

Lecture 3

Interpolation

Definition: Interpolation and extrapolation in one dimension

Given the set of data points x_i , $i = 1, \dots, n$, and the values $f(x_i)$ of a function $f(x)$ in these data points, the *interpolation* problem concerns with estimating (predicting, reconstructing) the values of $f(x)$ at a new point x , situated *between* the given data points. Whereas, the *extrapolation* problem deals with estimating $f(x)$ at a new point *outside* the interval covered by the old data points.

In *machine learning*, the pairs (x_i, f_i) are called the data points, and the problem is to *learn* the shape of the function $f(x)$, so that its value can be predicted for any new x .

Both the data points and the function values can be vectors, $\mathbf{x}_i \in \mathbb{R}^p$, $\mathbf{f}_i \in \mathbb{R}^m$, or even tensors.

linear interpolation

Linear interpolation

In linear interpolation the value of $f(x)$ at $x \in [x_0, x_1]$ is estimated/approximated as a value of the linear function $L(x)$ that goes through the data points $(x_0, f(x_0))$ and $(x_1, f(x_1))$.

A linear function has the general form:

$$L(x) = a + bx$$

and is completely determined by the two parameters a and b . Alternative expressions are possible and will be discussed in this Lecture.

Linear interpolation

Formally, we can pose the linear interpolation problem as follows. Find constants a and b such that:

$$L(x) = a + bx,$$

$$L(x_0) = f(x_0),$$

$$L(x_1) = f(x_1).$$

Solving:

$$(1) \begin{cases} a + bx_0 = f(x_0) \\ a + bx_1 = f(x_1) \end{cases} \quad (2) \begin{cases} a = f(x_0) - bx_0 \\ f(x_0) - bx_0 + bx_1 = f(x_1) \end{cases}$$

$$(3) \begin{cases} a = f(x_0) - bx_0 \\ b(x_1 - x_0) = f(x_1) - f(x_0) \end{cases} \quad (4) \begin{cases} a = f(x_0) - \frac{f(x_1) - f(x_0)}{x_1 - x_0} x_0 \\ b = \frac{f(x_1) - f(x_0)}{x_1 - x_0} \end{cases}$$

Linear interpolation

The solution is:

$$L(x) = f(x_0) - \frac{f(x_1) - f(x_0)}{x_1 - x_0}x_0 + \frac{f(x_1) - f(x_0)}{x_1 - x_0}x$$

Now, we rewrite it in a form that is easier to extend to other types of interpolating functions.

Linear Lagrange basis functions

Introducing linear *Lagrange basis functions* on the interval $[x_0, x_1]$:

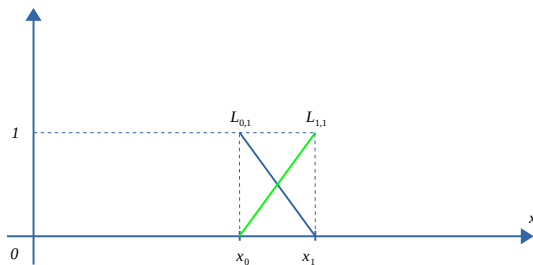
$$L_{0,1}(x) = \frac{x_1 - x}{x_1 - x_0},$$
$$L_{1,1}(x) = \frac{x - x_0}{x_1 - x_0}.$$

The second index in $L_{i,n}$, refers to the degree of the interpolation polynomial. It is $n = 1$ in the linear case (1st degree polynomial). The first index refers to the point where the basis function equals one (it equals zero in the other point):

$$L_{0,1}(x_0) = 1, \quad L_{0,1}(x_1) = 0,$$
$$L_{1,1}(x_0) = 0, \quad L_{1,1}(x_1) = 1.$$

Notice that $L_{0,1}$ is a linear decreasing function and $L_{1,1}$ is a linear increasing function.

Linear Lagrange basis functions



Finding the interpolator

The idea is to represent the function $L(x)$ via the linear combination of the functions $L_{0,1}(x)$ and $L_{1,1}(x)$, i.e., find coefficients c_0 and c_1 , such that:

$$\begin{aligned}L_1(x) &= c_0 L_{0,1}(x) + c_1 L_{1,1}(x), \\L_1(x_0) &= f(x_0), \quad L_1(x_1) = f(x_1).\end{aligned}$$

A linear combination of linear basis functions is a linear function (why?), and index 1 in $L_1(x)$ shows that it is a first-degree polynomial.

Using the very convenient properties of the basis functions we find:

$$\begin{cases} c_0 L_{0,1}(x_0) + c_1 L_{1,1}(x_0) = f(x_0), \\ c_0 L_{0,1}(x_1) + c_1 L_{1,1}(x_1) = f(x_1), \\ c_0 = f(x_0), \\ c_1 = f(x_1), \end{cases}$$

Definition: Linear Lagrange interpolation

The linear *Lagrange* interpolation of the function values $f(x_0)$ and $f(x_1)$ at the point $x \in [x_0, x_1]$ is defined as

$$L_1(x) = f(x_0)L_{0,1}(x) + f(x_1)L_{1,1}(x),$$

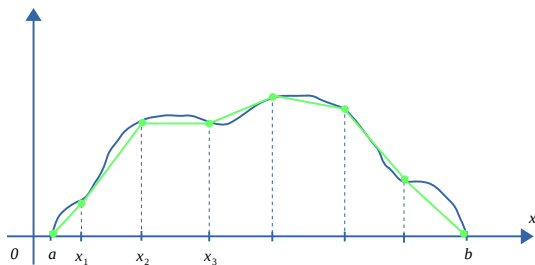
where the linear Lagrange basis functions are:

$$L_{0,1}(x) = \frac{x_1 - x}{x_1 - x_0}, \quad L_{1,1}(x) = \frac{x - x_0}{x_1 - x_0}.$$

It is easy to show that this equals the previously obtained expression

$$L(x) = f(x_0) - \frac{f(x_1) - f(x_0)}{x_1 - x_0}x_0 + \frac{f(x_1) - f(x_0)}{x_1 - x_0}x$$

Linearly interpolated function



approximation error of linear interpolation

How good is the estimate $L_1(x)$ of $f(x)$? The approximation error of linear interpolation is defined as

$$f(x) - L_1(x)$$

Obviously, this error is zero in the data points. It is also intuitively clear that a linear approximation of $f(x)$ introduces an error of the order $\mathcal{O}(x - x_0)^2$ or $\mathcal{O}(x - x_1)^2$. However, it is difficult to use the Taylor series technique here.

Theorem: Error of linear approximation

Let $x_0, x_1 \in [a, b]$, $x_0 < x_1$, and $f(x) \in C^2[a, b]$. If $L_1(x)$ is the linear polynomial, such that $L_1(x_0) = f(x_0)$ and $L_1(x_1) = f(x_1)$, then for each $x \in [a, b]$ there exists $\xi \in (a, b)$ such that:

$$f(x) - L_1(x) = \frac{f''(\xi)}{2}(x - x_0)(x - x_1).$$

Proof: error of linear approximation

We need to show the existence of $\xi \in (a, b)$ for any $x \in [a, b]$ such that the error formula holds. If $x = x_0$ or $x = x_1$, we have:

$$f(x) - L_1(x) = \frac{f''(\xi)}{2}(x - x_0)(x - x_1) = \frac{f''(\xi)}{2}(0) = 0,$$

which is true for any $f''(\xi)$ and therefore we can take any $\xi \in (a, b)$ and the formula will hold.

Suppose $x \neq x_0$ and $x \neq x_1$. For any such x , we can say that there is a number q such that

$$f(x) - L_1(x) = q(x - x_0)(x - x_1).$$

In fact, q is trivially given by

$$q = \frac{f(x) - L_1(x)}{(x - x_0)(x - x_1)},$$

but it is possible to show that $q = f''(\xi)/2$ for some $\xi \in (a, b)$ as well.

Proof: error of linear approximation

Looking at the difference function $\phi(t)$:

$$\phi(t) = f(t) - L_1(t) - q(t - x_0)(t - x_1),$$

we notice that: $\phi(x_0) = \phi(x) = \phi(x_1) = 0$, i.e. it takes the same value 0 at three points x_0 , x , and x_1 . By Rolle's Theorem, $\phi' = 0$ at some $t_1 \in (x_0, x)$ and $t_2 = (x, x_1)$. Since $\phi'(t_1) = \phi'(t_2) = 0$, by Rolle's Theorem, $\phi'' = 0$ at some $\xi \in (t_1, t_2) \subset (a, b)$, hence at some $\xi \in (a, b)$. Taking the second derivative of $\phi(t)$:

$$\phi''(t) = f''(t) - 2q.$$

For some $\xi \in (a, b)$ we have: $\phi''(\xi) = f''(\xi) - 2q = 0$. Therefore,

$$q = \frac{f''(\xi)}{2}.$$



Upper bound on the approximation error

We want a simple expression for the error in terms of the distance $|x_1 - x_0|$.

$$|f(x) - L_1(x)| = \left| \frac{f''(\xi)}{2} (x - x_0)(x - x_1) \right| \leq \frac{1}{2} \max_{\xi \in [a, b]} |f''(\xi)| |(x - x_0)(x - x_1)|$$

Now we show that

$$|(x - x_0)(x - x_1)| \leq \frac{1}{4} |x_1 - x_0|^2$$

Introducing the number a such that:

$$x - x_0 = a(x_1 - x_0),$$

we obtain

$$x_1 - x = (x_1 - x_0) - (x - x_0) = (x_1 - x_0) - a(x_1 - x_0) = (1 - a)(x_1 - x_0).$$

Hence,

$$|(x - x_0)(x - x_1)| = a(1 - a) |x_1 - x_0|^2$$

Upper bound on the approximation error

To show that

$$a(1-a) \leq \frac{1}{4},$$
$$-a^2 + a - \frac{1}{4} \leq 0,$$

consider the equation

$$-a^2 + a - \frac{1}{4} = 0.$$

It has only one solution:

$$a_{1,2} = \frac{-1 \pm \sqrt{1 - \frac{4}{4}}}{-2} = \frac{1}{2}$$

Taking the shape of the parabola into account, for all other values of a :

$$-a^2 + a - \frac{1}{4} \leq 0$$

Upper bound on the approximation error

$$|f(x) - L_1(x)| \leq \frac{1}{8}(x_1 - x_0)^2 \max_{\xi \in [a, b]} |f''(\xi)|$$

interpolation v.s. extrapolation

Theorem about the error allows to take $x \in [a, b]$, where $f \in C^2[a, b]$. It is possible that $[x_0, x_1] \subset [a, b]$ and $x \in [a, b]$, but $x \notin [x_0, x_1]$. When this happens we have *extrapolation* of $f(x)$ by $L_1(x)$ beyond the interval $[x_0, x_1]$ covered by the data points.

Does it mean that the errors of interpolation and extrapolation are really the same? Yes, the formula is the same. Although, $f''(\xi)$ is different for every new x . However, there is more at play here.

working with noisy data

Suppose that instead of the exact function values $f(x_0)$ and $f(x_1)$ we are given experimental data $\hat{f}(x_0)$ and $\hat{f}(x_1)$ such that:

$$|f(x_0) - \hat{f}(x_0)| \leq \epsilon,$$

$$|f(x_1) - \hat{f}(x_1)| \leq \epsilon,$$

where $\epsilon \geq 0$ is the upper bound on experimental error. If we use the experimental data to estimate $f(x)$ by the linear Lagrange interpolation, we get:

$$\hat{L}_1(x) = \hat{f}(x_0)L_{0,1}(x) + \hat{f}(x_1)L_{1,1}(x).$$

The error with respect to the interpolation of the clean data is:

$$\begin{aligned} L_1(x) - \hat{L}_1(x) &= (f(x_0) - \hat{f}(x_0))L_{0,1}(x) + (f(x_1) - \hat{f}(x_1))L_{1,1}(x) \\ |L_1(x) - \hat{L}_1(x)| &\leq |f(x_0) - \hat{f}(x_0)||L_{0,1}(x)| + |f(x_1) - \hat{f}(x_1)||L_{1,1}(x)| \\ &\leq \epsilon \frac{|x_1 - x|}{x_1 - x_0} + \epsilon \frac{|x - x_0|}{x_1 - x_0} = \frac{|x_1 - x| + |x - x_0|}{x_1 - x_0} \epsilon \end{aligned}$$

interpolation and extrapolation with noisy data

The bound

$$|L_1(x) - \hat{L}_1(x)| \leq \frac{|x_1 - x| + |x - x_0|}{x_1 - x_0} \epsilon$$

holds for both the interpolation, when $x \in [x_0, x_1]$ and the extrapolation, when $x \notin [x_0, x_1]$.

For interpolation we have the identity $|x_1 - x| + |x - x_0| = |x_1 - x_0|$ and the bound becomes:

$$|L_1(x) - \hat{L}_1(x)| \leq \epsilon, \quad x_0 \leq x \leq x_1$$

For extrapolation, e.g., $x > x_1$, we have $|x - x_0| = x_1 - x_0 + x - x_1$ and the bound becomes:

$$|L_1(x) - \hat{L}_1(x)| \leq \left(1 + 2 \frac{x - x_1}{x_1 - x_0}\right) \epsilon, \quad x > x_1$$

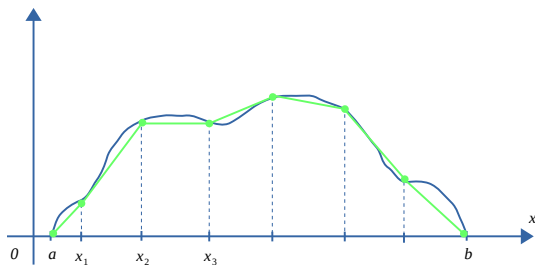
It grows linearly with x . A little noise in experimental data will produce very bad extrapolations. Extrapolation is *sensitive* with respect to noise.

Questions

- Derive the upper bound on the extrapolation error with noisy data for $x < x_0$.

higher-order Lagrange interpolation

can we improve on linear interpolation?



polynomial interpolation

- Suppose there are $n + 1$ distinct data points given, $x_0, \dots, x_n$, together with the $n + 1$ function values $f(x_0), \dots, f(x_n)$.
- It is always possible to fit the n -th degree polynomial function $L_n(x)$ so that:

$$L_n(x_j) = f(x_j), \quad j = 0, \dots, n$$

- Such a fitted polynomial could then be used to interpolate between the data points, i.e., to approximate the value of $f(x)$ by $L_n(x)$.

what does it mean, to fit a polynomial?

General expression for the n -th degree polynomial is:

$$L_n(x) = c_0 + c_1x + c_2x^2 + \cdots + c_nx^n = \sum_{j=0}^n c_jx^j.$$

Polynomial is defined by its $n + 1$ coefficients c_j , $j = 0, \dots, n$.

We want a special polynomial, such that fits the given function values at the data points:

$$L_n(x_j) = f(x_j), \quad j = 0, \dots, n$$

Hence, we need to find the coefficients that make this possible.

how to fit a polynomial?

It is a question of substituting the expression

$$L_n(x) = c_0 + c_1x + c_2x^2 + \cdots + c_nx^n = \sum_{j=0}^n c_jx^j.$$

into the conditions

$$L_n(x_i) = f(x_i), \quad i = 0, \dots, n$$

arriving at $n + 1$ algebraic equations for $n + 1$ coefficients:

$$c_0 + c_1x_0 + c_2x_0^2 + \cdots + c_nx_0^n = f(x_0),$$

$$\vdots$$

$$c_0 + c_1x_n + c_2x_n^2 + \cdots + c_nx_n^n = f(x_n),$$

algebraic problem of polynomial fitting

This can be written in the matrix form as:

$$\begin{bmatrix} 1 & x_0 & \dots & x_0^n \\ 1 & x_1 & \dots & x_1^n \\ \vdots & \vdots & \dots & \vdots \\ 1 & x_n & \dots & x_n^n \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_n) \end{bmatrix}$$

with the so-called Vandermonde matrix.

Hence, to find the coefficients c_j , $j = 0, \dots, n$, we need to solve this system, which is not easy for large n due to stability issues.

Lagrange form of polynomials

The same polynomial

$$L_n(x) = c_0 + c_1x + c_2x^2 + \cdots + c_nx^n = \sum_{j=0}^n c_jx^j.$$

can also be written in a different form:

$$L_n(x) = a_0L_{0,n} + a_1L_{1,n} + \cdots + a_nL_{n,n} = \sum_{k=0}^n a_kL_{k,n},$$

where a_k are the new coefficients and $L_{k,n}$ are the *Lagrange basis functions*:

$$L_{k,n}(x) = \frac{(x - x_0)(x - x_1) \cdots (x - x_{k-1})(x - x_{k+1}) \cdots (x - x_n)}{(x_k - x_0)(x_k - x_1) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)}$$

coefficients of the Lagrange polynomial

Lagrange basis functions have the remarkable property:

$$L_{k,n}(x_i) = \delta_{i,k} = \begin{cases} 1, & i = k, \\ 0, & i \neq k \end{cases}$$

Hence,

$$L_n(x_i) = \sum_{k=0}^n a_k L_{k,n}(x_i) = \sum_{k=0}^n a_k \delta_{i,k} = a_i$$

From the fitting conditions:

$$L_n(x_i) = f(x_i), \quad i = 0, \dots, n$$

we obtain:

$$a_i = f(x_i), \quad i = 0, \dots, n$$

Definition: Lagrange polynomial

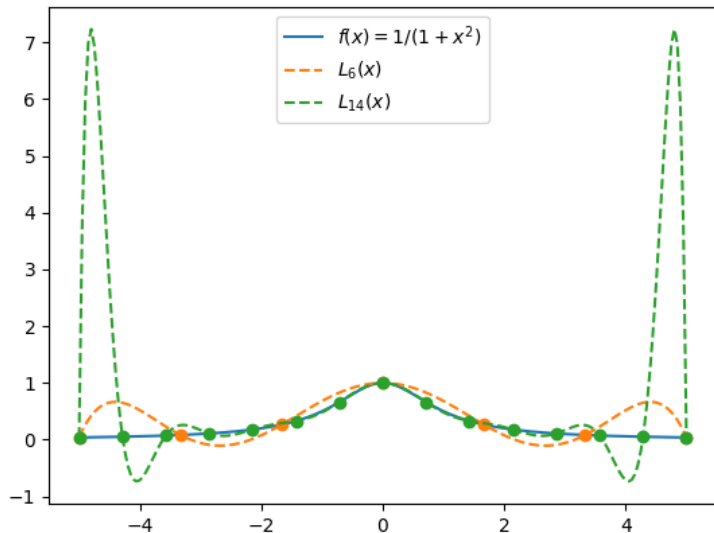
Given $n + 1$ distinct data points x_i and $n + 1$ function values $f(x_i)$, $i = 0, \dots, n$, the *Lagrange interpolating polynomial* is the n -th degree polynomial $L_n(x)$ that passes through all function values and has the form:

$$L_n = \sum_{j=0}^n f(x_j) L_{j,n}(x),$$

where the *Lagrange basis functions* are given by:

$$L_{k,n}(x) = \frac{(x - x_0)(x - x_1) \cdots (x - x_{k-1})(x - x_{k+1}) \cdots (x - x_n)}{(x_k - x_0)(x_k - x_1) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)}$$

Runge phenomenon



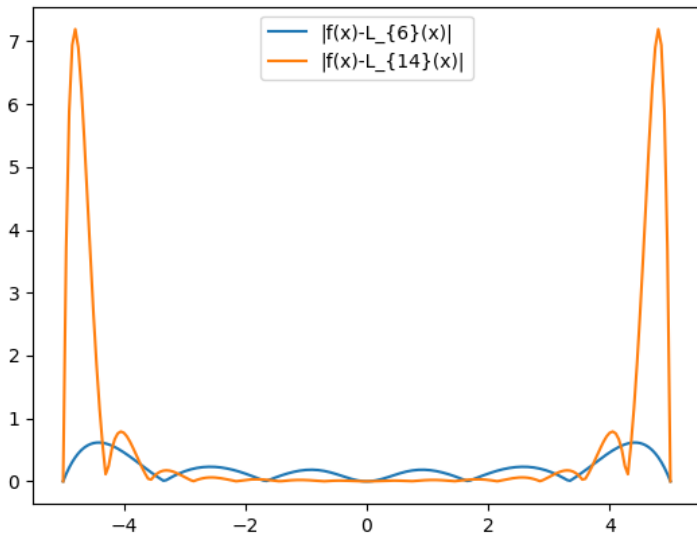
Theorem: Interpolation error

Let $x_0, \dots, x_n$ be the distinct nodes in $[a, b]$, $x_0 < x_1 < \dots < x_n$. Given the function $f \in C^{n+1}[a, b]$, let L_n be its Lagrange polynomial that equals f at the nodes. Then, for each $x \in [a, b]$ there exists $\xi \in (a, b)$ such that

$$f(x) - L_n(x) = (x - x_0) \cdots (x - x_n) \frac{f^{(n+1)}(\xi)}{(n+1)!}$$

Proof is similar to the linear (polynomial of degree one) case. This formula works also for extrapolation.

Interpolation error



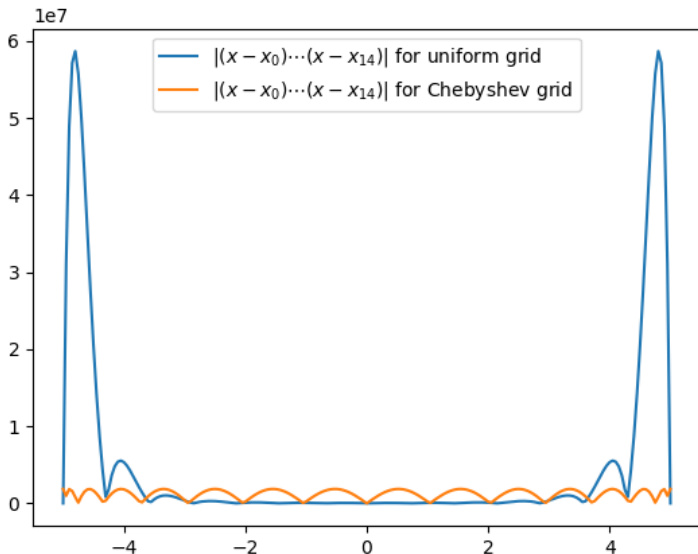
why error grows and what can be done about it?

$$f(x) - L_n(x) = (x - x_0) \cdots (x - x_n) \frac{f^{(n+1)}(\xi)}{(n+1)!}$$

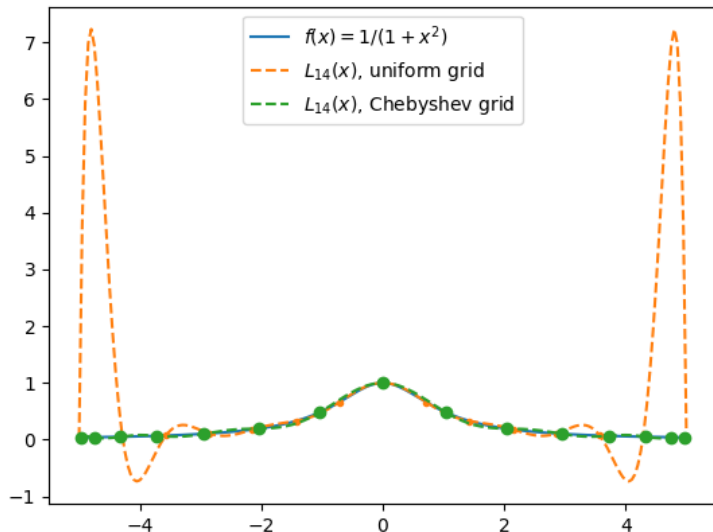
- There is nothing we can do about the factor $f^{(n+1)}(\xi)/(n+1)!$, and it is often not very large
- Apparently, the product $(x - x_0) \cdots (x - x_n)$ becomes larger at the ends of the interval, and it has to do with the uniform spacing between the data points x_i
- This product is uniformly minimized for all $x \in [a, b]$ by choosing the *Chebyshev grid* on $[a, b]$:

$$x_k = \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{2k+1}{2(n+1)}\pi\right), \quad k = 0, \dots, n$$

Optimizing the factor $|(x - x_0) \cdots (x - x_n)|$



Interpolating with Chebyshev nodes



spline interpolation

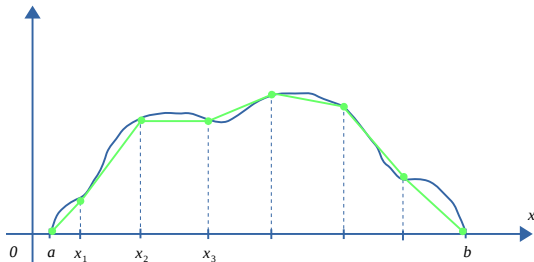
Summary of polynomial interpolation so far

- Higher-order polynomial interpolation can be done very efficiently with *Lagrange polynomials*
- Higher-order polynomial interpolation on a uniform grid of data points causes *Runge phenomenon*, where the error grows towards the ends of the interval
- The error can be minimized, if one uses the *Chebyshev grid* of data points
- what to do if you cannot change your data points?

Piecewise polynomials

- We want to avoid polynomials of very high degree, but still fit all $n + 1$ data points well
- Solution is to split the interval $[a, b]$ into sub-intervals, covered by small groups of data points and interpolate using separate lower-degree polynomials on each sub-interval
- Such polynomials defined on adjoint sub-intervals are called *piecewise polynomials*

- Piecewise polynomial of degree one, $s(x) = c_0 + c_1x$, has two parameters, both of which are used to fit $s(x)$ to the function values at the end of each sub-interval.
- Piecewise polynomial of degree two, $s(x) = c_0 + c_1x + c_2x^2$, has three parameters. Two will be used to fit the function values. What to do with the third parameter? Wrong choice may cause sharp corners at the nodes.



idea of splines

- Let there be a grid of $n + 1$ data points $x_0, \dots, x_n$ and $n + 1$ values of the function $f(x)$ at these points
- On each sub-interval $[x_k, x_{k+1}]$, we define a polynomial $s_k(x)$ of degree p
- In this way we define the spline of degree p on $[x_0, x_n]$ as

$$s(x) = \begin{cases} s_0(x), & x \in [x_0, x_1], \\ s_1(x), & x \in [x_1, x_2], \\ \vdots & \\ s_{n-1}(x), & x \in [x_{n-1}, x_n] \end{cases}$$

- We require that all $s_k(x)$ and their $p - 1$ derivatives are 'smoothly connected' at each node

Definition: Natural cubic spline of degree three

Let $x_0 < x_1 < \dots < x_n$ be the nodes in $[a, b]$ and $f(x_k)$, $k = 0, \dots, n$, the values of $f \in C[a, b]$. The *natural cubic spline of degree three* $s(x)$ has the following properties:

- 1 On each sub-interval $[x_k, x_{k+1}]$, $k = 0, \dots, n-1$; the spline $s(x)$ is a polynomial $s_k(x)$ of degree three.
- 2 At the nodes x_k , $k = 0, \dots, n$; the spline $s(x)$ satisfies:

$$\begin{aligned}s(x_k) &= f(x_k), \quad k = 0, \dots, n; \\s_k(x_{k+1}) &= s_{k+1}(x_{k+1}), \quad k = 0, \dots, n-2; \\s'_k(x_{k+1}) &= s'_{k+1}(x_{k+1}), \quad k = 0, \dots, n-2; \\s''_k(x_{k+1}) &= s''_{k+1}(x_{k+1}), \quad k = 0, \dots, n-2;\end{aligned}$$

- 3 At the boundary nodes, the *natural* boundary conditions are satisfied:

$$\begin{aligned}s''_0(x_0) &= 0, \\s''_{n-1}(x_n) &= 0.\end{aligned}$$

constructing a cubic spline

- We have the data points x_k , the function values $f(x_k)$, and use the notation: $f_k = f(x_k)$ and $h_k = x_{k+1} - x_k$
- We construct $s(x)$ by constructing its pieces $s_k(x)$ on all sub-intervals $[x_k, x_{k+1}]$
- It is common to represent the third-degree $s_k(x)$ as

$$s_k(x) = a_k(x - x_k)^3 + b_k(x - x_k)^2 + c_k(x - x_k) + d_k$$

- To find $s_k(x)$ we need to find its four coefficients a_k, b_k, c_k, d_k
- To find the coefficients we use the four conditions from the definition of the natural cubic spline

constructing a cubic spline: d_k

The first condition is:

$$s_k(x_k) = f_k, \quad k = 0, \dots, n;$$

Substituting here the expression for $s_k(x)$, we get:

$$a_k(x_k - x_k)^3 + b_k(x_k - x_k)^2 + c_k(x_k - x_k) + d_k = f_k,$$

or

$$d_k = f_k, \quad k = 0, \dots, n$$

constructing a cubic spline: a_k

The fourth condition is:

$$s_k''(x_{k+1}) = s_{k+1}''(x_{k+1}), \quad k = 0, \dots, n-2;$$

Substituting here the expression for $s_k(x)$, we get:

$$\begin{aligned} 6a_k(x_{k+1} - x_k) + 2b_k &= 6a_{k+1}(x_{k+1} - x_{k+1}) + 2b_{k+1}, \\ 6h_k a_k + 2b_k &= 2b_{k+1} \end{aligned}$$

or

$$a_k = \frac{1}{3h_k}(b_{k+1} - b_k), \quad k = 0, \dots, n-2$$

constructing a cubic spline: c_k

The second condition is:

$$s_k(x_{k+1}) = s_{k+1}(x_{k+1}), \quad k = 0, \dots, n-2;$$

Substituting here the expression for $s_k(x)$, we get:

$$\begin{aligned} & a_k(x_{k+1} - x_k)^3 + b_k(x_{k+1} - x_k)^2 + c_k(x_{k+1} - x_k) + d_k \\ &= a_{k+1}(x_{k+1} - x_{k+1})^3 + b_{k+1}(x_{k+1} - x_{k+1})^2 + c_{k+1}(x_{k+1} - x_{k+1}) + d_{k+1}, \\ & h_k^3 a_k + h_k^2 b_k + h_k c_k + d_k = d_{k+1}, \\ & \frac{h_k^3}{3h_k} (b_{k+1} - b_k) + h_k^2 b_k + h_k c_k + f_k = f_{k+1} \end{aligned}$$

or

$$c_k = \frac{f_{k+1} - f_k}{h_k} - \frac{h_k}{3}(2b_k + b_{k+1}), \quad k = 0, \dots, n-2$$

constructing a cubic spline: b_k

The third condition is:

$$s'_k(x_{k+1}) = s'_{k+1}(x_{k+1}), \quad k = 0, \dots, n-2;$$

Substituting here the expression for $s_k(x)$, we get:

$$\begin{aligned} & 3a_k(x_{k+1} - x_k)^2 + 2b_k(x_{k+1} - x_k) + c_k \\ &= 3a_{k+1}(x_{k+1} - x_{k+1})^2 + 2b_{k+1}(x_{k+1} - x_{k+1}) + c_{k+1}, \\ & 3h_k^2 a_k + 2h_k b_k + c_k = c_{k+1}, \\ & \frac{3h_k^2}{3h_k}(b_{k+1} - b_k) + 2h_k b_k + \frac{f_{k+1} - f_k}{h_k} - \frac{h_k}{3}(2b_k + b_{k+1}) \\ &= \frac{f_{k+2} - f_{k+1}}{h_{k+1}} - \frac{h_{k+1}}{3}(2b_{k+1} + b_{k+2}), \\ & h_k b_{k+1} - h_k b_k + 2h_k b_k - \frac{2}{3}h_k b_k - \frac{1}{3}h_k b_{k+1} + \frac{2}{3}h_{k+1}b_{k+1} + \frac{1}{3}h_{k+1}b_{k+2} \\ &= \frac{f_{k+2} - f_{k+1}}{h_{k+1}} - \frac{f_{k+1} - f_k}{h_k} \end{aligned}$$

constructing a cubic spline: b_k

$$\begin{aligned} & \left(-1h_k + 2h_k - \frac{2}{3}h_k\right) b_k + \left(h_k - \frac{1}{3}h_k + \frac{2}{3}h_{k+1}\right) b_{k+1} + \frac{1}{3}h_{k+1}b_{k+2} \\ &= \frac{f_{k+2} - f_{k+1}}{h_{k+1}} - \frac{f_{k+1} - f_k}{h_k}, \\ & \frac{1}{3}h_k b_k + \frac{2}{3}(h_k + h_{k+1})b_{k+1} + \frac{1}{3}h_{k+1}b_{k+2} = \frac{f_{k+2} - f_{k+1}}{h_{k+1}} - \frac{f_{k+1} - f_k}{h_k} \\ & h_k b_k + 2(h_k + h_{k+1})b_{k+1} + h_{k+1}b_{k+2} = 3 \left(\frac{f_{k+2} - f_{k+1}}{h_{k+1}} - \frac{f_{k+1} - f_k}{h_k} \right), \\ & k = 0, \dots, n-2 \end{aligned}$$

constructing a cubic spline: b_k

Now we use the natural boundary conditions:

$$s_0''(x_0) = 0, \quad s_{n-1}''(x_n) = 0.$$

Substituting here the expression for $s_k(x)$, we get

$$6a_0(x_0 - x_0) + 2b_0 = 0,$$

$$b_0 = 0$$

and

$$6a_{n-1}(x_n - x_{n-1}) + 2b_{n-1} = 0,$$

$$6h_{n-1}a_{n-1} + 2b_{n-1} = 0,$$

$$\frac{6h_{n-1}}{3h_{n-1}}(b_n - b_{n-1}) + 2b_{n-1} = 0,$$

$$2b_n = 0,$$

$$b_n = 0$$

constructing a cubic spline: b_k

$$b_0 = 0,$$

$$h_k b_k + 2(h_k + h_{k+1})b_{k+1} + h_{k+1}b_{k+2} = 3 \left(\frac{f_{k+2} - f_{k+1}}{h_{k+1}} - \frac{f_{k+1} - f_k}{h_k} \right),$$
$$k = 0, \dots, n-2,$$

$$b_n = 0,$$

is a system of linear algebraic equations that is solved numerically to get all b_k , $k = 1, \dots, n-1$. After that all other coefficients of all $s_k(x)$ can be computed explicitly.

Questions

- Given three data points, x_0, x_1, x_2 , and three function values, $f(x_0), f(x_1), f(x_2)$, write down the Lagrange polynomial explicitly.
- Write down the Lagrange basis function $L_{2,4}(x)$ explicitly.
- Suppose you can 'sample' some function $f \in C^{n+1}$ at $n+1$ nodes of your choice. Which interpolation method would you chose and why?
- Suppose you are given $n+1$ fixed nodes and $n+1$ values of an unknown function $f(x)$ at these nodes. Which interpolation method would you chose and why?
- What conditions must be satisfied by the cubic splines?